Команда «Аделинчики»

Нейропоиск

Умный поиск по книгам с помощью искусственного интеллекта

Отборочное задание направления «Искусственный интеллект» Кейс: «Умный поиск по книгам»

Наша команда

Совместный проект на Python

Untitled - Notepad

File Edit Search Help

- Алексей Котов
- Глеб Христофоров
- Богдан Дубовицкий
- Эдуард Сафронов

Цели и задачи

Цель:
Создать интеллектуальную систему для работы с текстами книг, способную понимать смысл и контекст, а не просто искать совпадения слов.

- Находить релевантные фрагменты в книгах по смысловому запросу

- Отвечать на вопросы по содержанию текста с точными цитатами

Задачи:
- Загрузка файлов и папок с книгами различных форматов

- Поиск — возврат 3–5 фрагментов с указанием источника

- Вопросы — ответ с дословными цитатами

- Честное сообщение об отсутствии информации

- Удобный светлый GUI без установки

Python 3.14.0 — Стек технологий

Библиотека	Назначение
customtkinter	Светлый графический интерфейс
sentence-transformers	Векторные эмбеддинги, модель all-mpnet-base-v2, работает на CPU
piragi (LanceDB)	Локальная векторная база данных, индекс в %TEMP%\NeuroPoiskov
pymupdf, python-docx, ebooklib, chardet	Чтение 15+ форматов файлов
OpenRouter API	Бесплатные языковые модели, temperature=0.1, max_tokens=3000

```python
try:
    import customtkinter as ctk
    import tkinter as _tk
except ImportError:
    sys.exit(1)

ctk.set_appearance_mode("light")
ctk.set_default_color_theme("dark-blue")
```

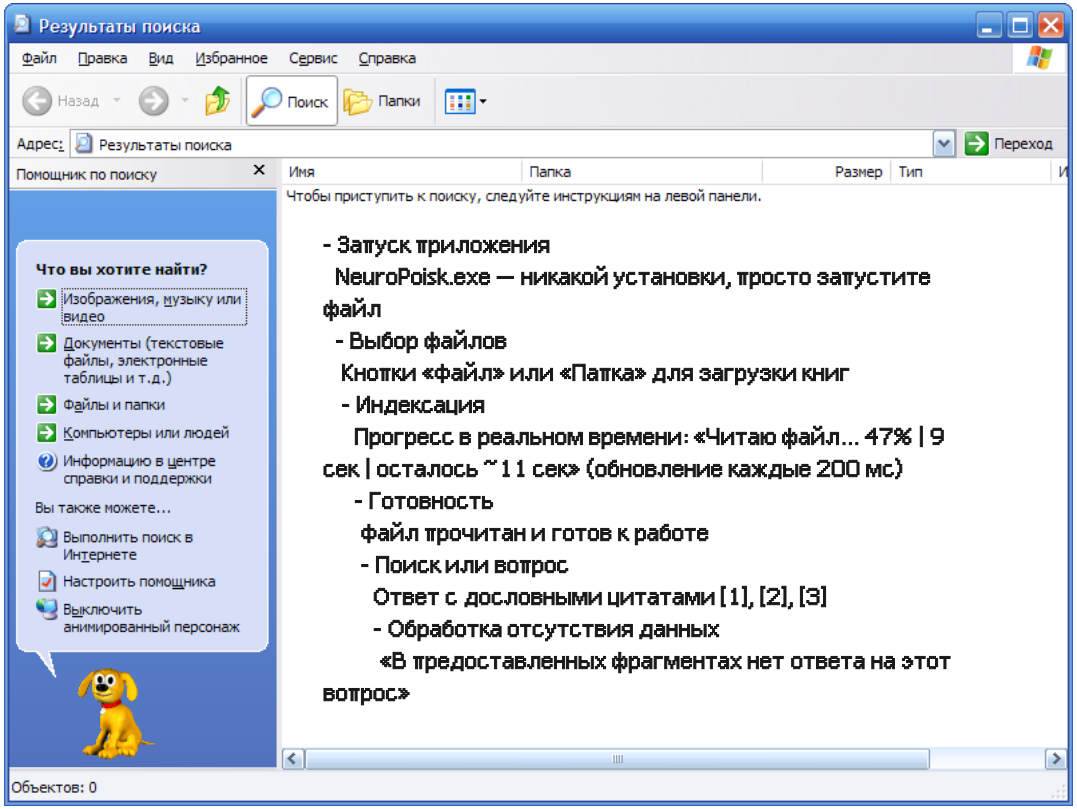
```python
t_start = time.time()
kb = Ragi(str(fp), persist_dir=str(index_dir), config={
    "embedding": {"model": "all-mpnet-base-v2", "device": EMBED_DEVICE},
})
```

```python
def make_kb(docs_path, progress):
    from piragi import Ragi
    fp  = Path(docs_path)
    NATIVE_EXTS = {".txt", ".md", ".pdf", ".docx", ".csv"}
```

```python
def _read_epub(fp):
    try:
        import ebooklib
        from ebooklib import epub
        from bs4 import BeautifulSoup
        book  = epub.read_epub(str(fp))
        parts = [BeautifulSoup(item.get_content(), "lxml").get_text("\n")
```

```python
try:
    req = urllib.request.Request(
        "https://openrouter.ai/api/v1/models",
        headers={"Authorization": f"Bearer {OPENROUTER_API_KEYS[0]}"},
    )
```

Как это выглядит в работе



Примеры запросов

Поиск фрагментов

- Найди где говорится про первую встречу Мастера и Маргариты

- Покажи фрагмент про бал у Воланда

- Найди все места где упоминается Понтий Пилат

Вопросы по содержанию

- Почему Раскольников решился на убийство?

- Что произошло с Мастером в конце книги?

- Чем заканчивается книга?

Общие вопросы о книге

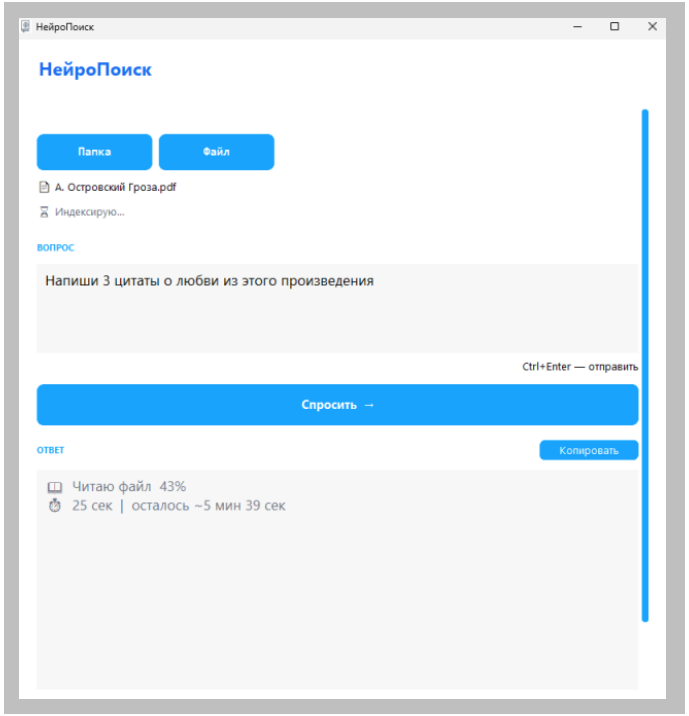
- О чём эта книга?

- Кто главный герой и каков его характер?

- Какова главная мысль произведения?

На все запросы — ответ строго по тексту, с дословными цитатами [1][2][3], честно при отсутствии информации.

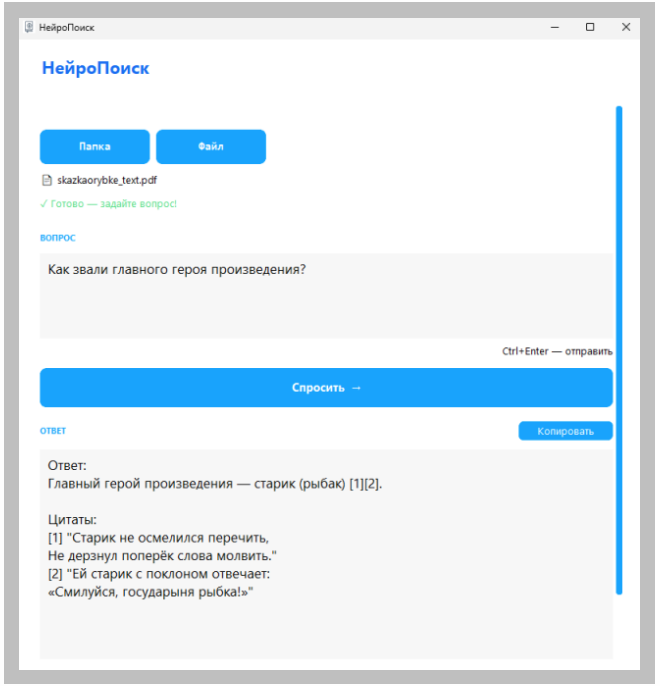
Интерфейс приложения и его функционал:



Приложение позволяет вставить файл или папку, чтобы бот прочитал текст произведения и мог ответить на вопросы в поле, которые вы впишете.

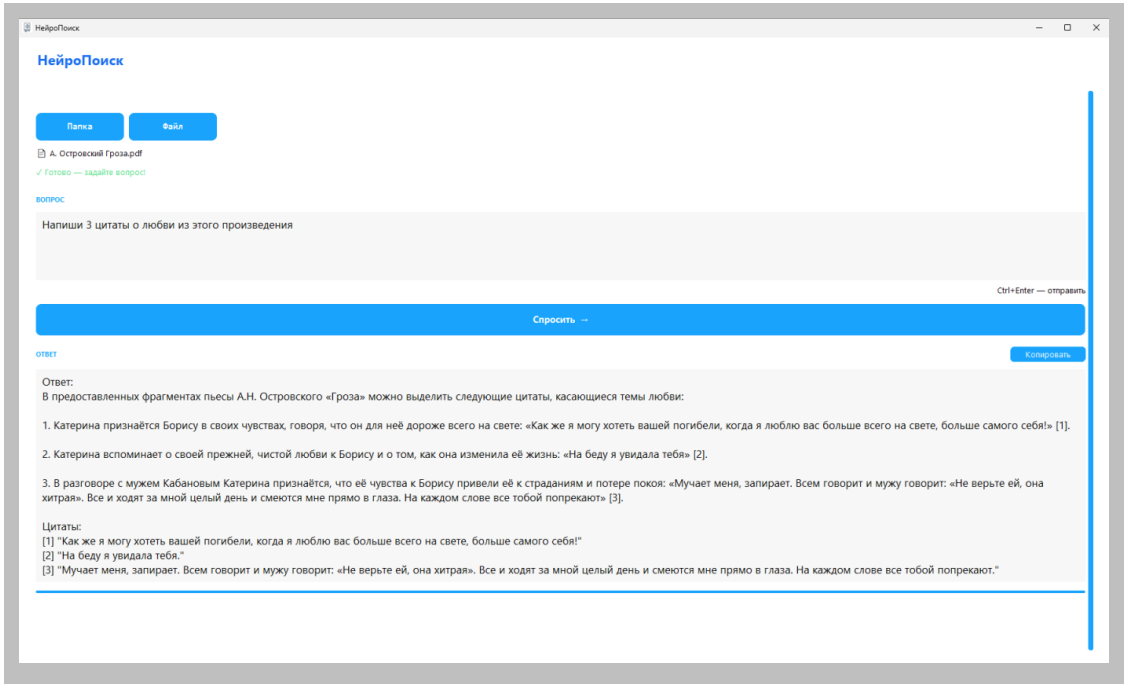
После отправки текста в виде документа начинается его обработка.

Интерфейс приложения и его функционал:



После обработки файла нужно нажать кнопку «Спросить», после чего бот ответит на ваш вопрос исходя из текста.

Интерфейс приложения и его функционал:



Приложение также позволяет вычленять из текста цитаты, которые соответствуют содержанию вашего запроса.

Архитектура: реальный код

функция retrieve_chunks

Векторный поиск + дополнительные литературные запросы для лучшего охвата

```
def retrieve_chunks(kb, question):
    seen, chunks = set(), []
    def add(cs):
        for c in cs:
            t = c.text if hasattr(c, "text") else str(c)
            k = t[:120].strip()
            if k not in seen:
                seen.add(k)
                chunks.append(c)
    add(_raw_retrieve(kb, question))
    for q in _LIT_QUERIES:
        if len(chunks) >= TOP_K:
            break
        add(_raw_retrieve(kb, q))
    return chunks[:TOP_K]
```

Системный промт модели

Отвечай только на русском языке. Используй исключительно информацию из контекста. НЕ придумывай факты. Если в контексте нет ответа — напиши: "В предоставленных фрагментах нет ответа на этот вопрос". ВСТАВЛЯЙ ссылки [1], [2], [3] внутри ответа. В конце приведи список дословных цитат.
temperature=0.1, max_tokens=3000

Преимущества системы

- Полностью соответствует заданию
  Поиск, вопросы, честный ответ при отсутствии данных

- 15+ форматов
  .txt .fb2 .epub .mobi .azw3 .djvu .pdf .docx .rtf .odt .html .md и другие

- Готовый .exe
  Собран через PyInstaller, Python устанавливать не нужно

- Поиск по смыслу, не Ctrl+F
  Модель all-mpnet-base-v2, семантические эмбеддинги

- Индексация локально
  Только найденные фрагменты уходят в OpenRouter

- Бесплатно
  Используются только бесплатные модели OpenRouter (:free)

```python
if not context:
    return "⚠ Не удалось получить текст из документов."
return ask_openrouter(context, question)
```

```python
SUPPORTED_EXTS = {
    ".txt", ".md", ".pdf", ".docx", ".csv",
    ".fb2", ".epub", ".mobi", ".azw3", ".azw",
    ".djvu", ".djv", ".rtf", ".odt", ".html", ".htm",
}
```

```python
if getattr(sys, "frozen", False):
    BASE_DIR = Path(sys.executable).parent
else:
    BASE_DIR = Path(__file__).parent
```

```python
def retrieve_chunks(kb, question):
    seen, chunks = set(), []
    def add(cs):
        for c in cs:
            t = c.text if hasattr(c, "text") else str(c)
            k = t[:120].strip()
            if k not in seen:
                seen.add(k)
                chunks.append(c)
    add(_raw_retrieve(kb, question))
    for q in _LIT_QUERIES:
        if len(chunks) >= TOP_K:
            break
        add(_raw_retrieve(kb, q))
    return chunks[:TOP_K]
```

```python
def fetch_free_models():
    import urllib.request, json
    global _models_cache, _models_cache_time
    if _models_cache and (time.time() - _models_cache_time) < _MODELS_TTL:
        return _models_cache
    try:
        req = urllib.request.Request(
            "https://openrouter.ai/api/v1/models",
            headers={"Authorization": f"Bearer {OPENROUTER_API_KEYS[0]}"},
        )
        with urllib.request.urlopen(req, timeout=10) as r:
            data = json.loads(r.read())
        models = [m["id"] for m in data.get("data", []) if m.get("id", "").endswith(":free")]
```

```python
index_dir = Path(tempfile.gettempdir()) / "NeuroPoiskov"
index_dir.mkdir(parents=True, exist_ok=True)

t_start = time.time()
kb = Ragi(str(fp), persist_dir=str(index_dir), config={
    "embedding": {"model": "all-mpnet-base-v2", "device": EMBED_DEVICE},
})
```

Спасибо за внимание!

НейроПоиск — умный помощник для работы с литературой